

1. Real-Time Face Detection and Recognition

Aim

To detect multiple faces from a live camera, recognize registered persons, and display their names with bounding boxes in real time.

Computer Vision Flow

Camera → Image Acquisition → Preprocessing → Face Detection → Feature Extraction → Face Recognition → Display Result

Pseudocode

START

1. Import OpenCV and required libraries.

2. Load the trained face recognition model.

3. Load the registered face dataset.

For each registered person:

Read the person's face images.

Extract facial features.

Store the features along with the person's name.

4. Open the camera using VideoCapture.

5. Check whether the camera is opened successfully.

If camera is not available:

Display "Camera not found"

STOP

6. WHILE camera is running:

a. Read one frame from the camera.

b. If the frame is not received:

BREAK the loop.

c. Resize the frame to a smaller size

to improve processing speed.

d. Convert the frame from BGR to grayscale.

e. Apply basic preprocessing if required

to reduce noise and improve image quality.

f. Detect all faces present in the frame.

g. FOR each detected face:

i. Get the face coordinates:

x, y, width, height

ii. Crop the face from the frame.

iii. Resize the cropped face

to the required input size.

iv. Extract facial features

from the detected face.

v. Compare the extracted features

with the registered face features.

vi. Find the closest matching person.

vii. IF the matching score is above

the selected threshold:

Set label = person's name

ELSE:

Set label = "Unknown"

viii. Draw a rectangle around the face.

ix. Display the person's name
above the rectangle.

h. Display the processed frame
using imshow().

i. Check the keyboard for the exit key.

j. IF the exit key is pressed:
BREAK the loop.

7. Release the camera.

8. Close all OpenCV windows.

STOP

Explanation of the Main Steps

Image Acquisition:

VideoCapture() continuously receives frames from the camera.

Preprocessing:

The frame is resized and converted to grayscale. This reduces the amount of data that has to be processed.

Face Detection:

OpenCV face detection is used to find all faces in the frame. Each detected face gets a bounding box.

Feature Extraction:

Features are extracted from every detected face. These features represent important information about the person's face.

Recognition:

The extracted features are compared with the registered dataset. If a suitable match is found, the person's name is displayed. Otherwise, the label is **Unknown**.

Visualization:

A rectangle is drawn around every detected face and the identity is displayed above it.

Real-Time Optimization

To reduce processing delay:

- Resize the camera frame before detection.
- Process only the required face regions.
- Use a lightweight face detector.
- Avoid loading the recognition dataset inside the loop.
- Detect faces at suitable intervals instead of unnecessarily repeating expensive operations.
- Use tracking between detections when appropriate.
- Release unused resources.

Output:

```
+-----+
|          |
| [Face]    |
| LOGESH    |
|          |
|   [Face]  |
| UNKNOWN   |
|          |
+-----+
```

2. Vehicle Detection, Tracking and Counting

Aim

To detect vehicles from a live traffic video, track them across frames, count them when they cross a virtual line, and display their IDs and total count.

Computer Vision Flow

Camera → Preprocessing → Vehicle Detection → Tracking → Line Crossing → Counting → Visualization

Pseudocode

START

1. Import OpenCV and required libraries.
2. Open the traffic video or live camera
using VideoCapture.
3. Load the vehicle detection method/model.
4. Initialize the vehicle tracker.
5. Set the Region of Interest (ROI).
6. Draw a virtual counting line
inside the traffic region.
7. Set vehicle_count = 0.
8. Create an empty list/set called
counted_ids to store vehicles
that have already been counted.
9. WHILE the video is running:

- a. Read the next video frame.
- b. If the frame is not available:
 BREAK the loop.
- c. Resize the frame if required
 to improve processing speed.
- d. Apply preprocessing such as
 noise reduction if required.
- e. Select the required Region of Interest.
- f. Detect vehicles inside the ROI.
- g. FOR each detected vehicle:
 - i. Get the bounding box:
 x, y, width, height
 - ii. Find the centre point
 of the vehicle.
 - iii. Give the vehicle a tracking ID.
 - iv. Match the vehicle with its
 position in the previous frame.
 - v. Update the vehicle's position.
- h. FOR each tracked vehicle:
 - i. Compare its current centre

with its previous centre.

ii. Check whether the vehicle
has crossed the virtual line.

iii. IF the vehicle has crossed
the line AND its ID is not
present in counted_ids:

```
vehicle_count =  
vehicle_count + 1
```

Add vehicle ID
to counted_ids.

i. Draw a rectangle around every vehicle.

j. Display the tracking ID
near each vehicle.

k. Draw the virtual counting line.

l. Display:

```
"Vehicle Count = vehicle_count"
```

m. Show the processed frame.

n. Check whether the exit key is pressed.

o. IF the exit key is pressed:

BREAK the loop.

10. Release the video/camera.

11. Close all OpenCV windows.

STOP

Explanation of the Main Steps

1. Video Acquisition

The traffic camera provides frames continuously using OpenCV VideoCapture().

Each frame is processed one after another.

2. Preprocessing

The frame can be resized and filtered to remove unnecessary noise.

This improves processing speed and makes vehicle detection easier.

3. Region of Interest

Instead of processing the complete image, only the road area is selected.

For example:

```
+-----+
|      Background      |
|                      |
| +-----+ |
| |  ROAD / ROI  | |
| |  Car   Car   Car | |
| +-----+ |
|      -----      |
|      LINE          |
+-----+
```

4. Vehicle Detection

The system identifies vehicles such as:

- Cars
- Buses
- Trucks

- Motorcycles

Each detected vehicle receives a bounding box.

5. Tracking

The same vehicle must be identified across consecutive frames.

For example:

Frame 1 → Vehicle ID 1

Frame 2 → Vehicle ID 1

Frame 3 → Vehicle ID 1

Frame 4 → Vehicle ID 1

6. Counting

A virtual line is drawn on the road.

The centre point of each vehicle is monitored.

If:

Previous position → Line → Current position

then the vehicle is considered to have crossed the line.

The system increases the count only if that vehicle ID has not already been counted.

Avoiding Double Counting

This is the important part of the problem.

The system maintains:

counted_ids = {2, 5, 8}

If vehicle ID 5 crosses the line again:

ID 5 already counted

→ Do not increase count

Visualization

The final video displays:

Vehicle ID: 1

Vehicle ID: 2

Vehicle ID: 3

----- ← Virtual line

Vehicle Count: 3

Real-Time Optimization

For near real-time operation:

- Resize frames.
- Process only the ROI.
- Use an efficient vehicle detector.
- Track vehicles instead of detecting unnecessarily many times.
- Avoid storing unnecessary frames.
- Use simple centre-point calculations for line crossing.
- Stop processing immediately when the exit key is pressed.